

Avaliação Oficial 01 - 2026.2

Desenvolvimento de Chatbot

David Capulot Corrêa RA: 399520311025

Gabriel do Almo Silveira Vicente RA: 400140711025

Peter Emmerich Mulim e Silva RA: 133354411025

Welington Carlos Silva De Barros RA: 381664411025

Centro Universitário Anhanguera de Niterói

Resumo. *Este relatório fala sobre o desenvolvimento do Pedro, um chatbot textual feito em Python que atende clientes de uma loja de produtos do Fluminense. O sistema limpa o texto digitado, reconhece a intenção do cliente usando um classificador Naive Bayes multinomial (scikit-learn) e devolve respostas por meio de estrutura de decisão e de um pequeno mecanismo de contexto. Aqui mostraremos as decisões do nosso projeto, o processamento de texto, exemplos de execução e as limitações da solução.*

1. Introdução

O objetivo do nosso sistema é simular um atendente virtual de uma loja do Fluminense. O Pedro conversa com o cliente no terminal e responde a dúvidas sobre os produtos vendidos, os preços, as formas de pagamento e a entrega. Na nossa loja fictícia, tudo é presencial: o pagamento e a retirada são feitos no local, e não há entrega.

O funcionamento geral é um ciclo. O programa pergunta o nome do cliente e entra em um laço de repetição, no qual lê cada mensagem, descobre a intenção dela, monta uma resposta e a exibe na tela. O laço só termina quando o cliente se despede. Para reconhecer as intenções, treinamos um classificador de Aprendizado de Máquina. Para escolher as respostas, usamos regras que consultam um catálogo de produtos.

2. Decisões de Projeto

Dividimos o problema em duas perguntas: o que o cliente quer e sobre o que ele está falando. A primeira é respondida pelo modelo de Aprendizado de Máquina, que classifica a mensagem em uma de nove intenções: saudação, ajuda, informação, produto, preço, pagamento, entrega, despedida e desconhecida. A segunda é respondida por regras, que procuram na mensagem o nome de um produto ou de um modelo, como "infantil" ou "goleiro". Separamos as duas tarefas porque o classificador é bom para reconhecer o tipo de pergunta, mas não sabe nada sobre preços. Isso fica no catálogo. Escolhemos cada estrutura pensando no que ela precisava fazer:

- *Dicionário de exemplos*: cada intenção aponta para uma lista de frases de treino. Assim, uma frase nunca fica sem o seu rótulo, o que poderia acontecer se mantivéssemos duas listas separadas.
- *Listas de respostas*: cada intenção tem várias respostas prontas, e o programa sorteia uma delas. Fizemos isso para o Pedro não repetir sempre a mesma frase, principalmente nas respostas sobre pagamento e entrega, que sempre repetem a mesma ideia.
- *Dicionário aninhado para o catálogo*: cada produto tem uma descrição e um preço ou, no caso das camisas e dos shorts, um dicionário de modelos. Adicionar um produto novo exige apenas uma entrada nova no dicionário, sem mexer nas regras.
- *Dicionário de apelidos*: liga palavras alternativas ao nome oficial do produto, como "bermuda" e "calcao" para short.
- *Conjunto de palavras ignoradas*: usamos um conjunto porque a consulta de pertencimento é rápida e não precisamos de ordem nem de repetição.
- *Tupla de contexto*: guarda o último produto mencionado, para o programa entender perguntas como "e a infantil?".
- *DataFrame do pandas*: organiza as frases de treino e suas intenções antes de transformá-las em números.

O laço principal segue sempre a mesma sequência: ler a entrada, prever a intenção, gerar a resposta com base na intenção e no contexto, exibir a resposta e verificar se a intenção é de despedida. Na geração da resposta, a ordem das verificações é a seguinte. Primeiro, procuramos um modelo específico do produto e, se ele existir, respondemos apenas com o preço dele. Depois, se o cliente citar um produto, mostramos a descrição e todos os preços de uma vez. Por último, se não há produto algum, sortecemos uma resposta da lista de intenção. Em todos os casos o programa atualiza o contexto, exceto quando a intenção é de despedida, em que o contexto é ignorado.

3. Processamento de Texto

```
def preparar_texto(texto):
    texto = normalizar(texto)
    texto = remover_palavras_ignoradas(texto)
    return texto
```

Antes de qualquer coisa, tanto as mensagens dos usuários quanto os exemplos de treino passam pela função `preparar_texto`, que tem duas etapas. Na primeira, a `normalizar` deixa tudo em minúsculas, troca a pontuação por espaços, tira os espaços duplicados e remove os acentos com a normalização Unicode.

```
def normalizar(texto):

    texto = texto.lower().strip()

    texto = texto.replace(".", " ")
    texto = texto.replace(",", " ")
    texto = texto.replace("; ", " ")
    texto = texto.replace(":", " ")
    texto = texto.replace("!", " ")
    texto = texto.replace("?", " ")
    texto = texto.replace("(", " ")
    texto = texto.replace(")", " ")
    texto = texto.replace("[", " ")
    texto = texto.replace("]", " ")
    texto = texto.replace("{", " ")
    texto = texto.replace("}", " ")
    texto = texto.replace("/", " ")
    texto = texto.replace("\\", " ")
    texto = texto.replace("-", " ")
    texto = texto.replace("_", " ")
    texto = texto.replace("'", "'")
    texto = texto.replace('"', '"')
    texto = texto.replace("@", "@")
```

```

texto = texto.replace("#", "")

texto = texto.replace("%", "")

texto = " ".join(texto.split())

texto = unicodedata.normalize("NFD", texto)

texto = "".join(
    c for c in texto
    if unicodedata.category(c) != "Mn"
)

return texto

```

Na segunda, a `remover_palavras_ignoradas` separa o texto em palavras e tira as que pouco ajudam a identificar o assunto, como artigos, preposições e pronomes ("a", "de", "para", "com").

```

def remover_palavras_ignoradas(texto):
    palavras = texto.split()

    palavras_filtradas = [
        palavra
        for palavra in palavras
        if palavra not in palavras_ignorar
    ]
    if not palavras_filtradas:
        return texto

    return " ".join(palavras_filtradas)

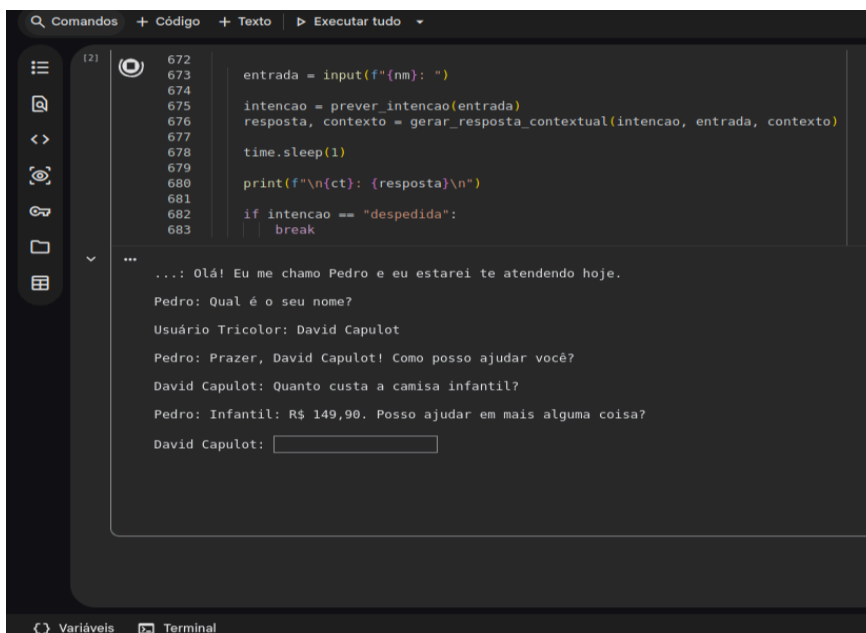
```

Caso o texto fique vazio depois dessa limpeza, usamos o original.

Para o computador conseguir trabalhar com o texto, convertemos cada mensagem em números usando o **CountVectorizer**. Ele simplesmente conta quantas vezes cada palavra do vocabulário aparece na mensagem, a chamada abordagem de bag of words, que ignora a ordem em que as palavras aparecem. Em cima dessa contagem usamos o Naive Bayes multinomial (**MultinomialNB**), que estima a probabilidade de cada intenção e fica com a mais provável. O treino acontece toda vez que o programa é iniciado. Se nenhuma das palavras da mensagem existir no vocabulário, nem chamamos o modelo, a intenção é tratada direto como desconhecida.

4. Exemplos de Execução

Interação 1



The screenshot shows a code editor with a dark theme. The top bar has tabs for 'Comandos', 'Código', 'Texto', and 'Executar tudo'. The 'Código' tab is active, showing Python code from line 672 to 683. The code defines a function to generate a response based on user input. The 'Executar tudo' button is visible. Below the code, the execution output is displayed, showing a conversation between a user and a chatbot named Pedro. The chatbot identifies the product 'camisa' and the model 'infantil' and returns the price R\$ 149,90.

```
672 entrada = input(f"{nm}: ")
673
674
675 intencao = prever_intencao(entrada)
676 resposta, contexto = gerar_resposta_contextual(intencao, entrada, contexto)
677
678 time.sleep(1)
679
680 print(f"\n{ct}: {resposta}\n")
681
682 if intencao == "despedida":
683     break
```

...: Olá! Eu me chamo Pedro e eu estarei te atendendo hoje.

Pedro: Qual é o seu nome?

Usuário Tricolor: David Capulot

Pedro: Prazer, David Capulot! Como posso ajudar você?

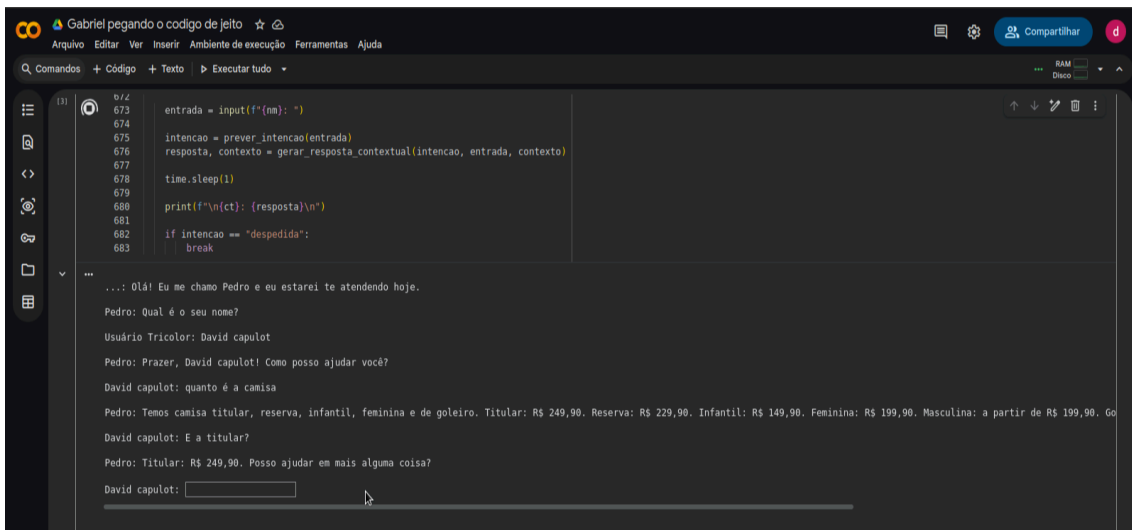
David Capulot: Quanto custa a camisa infantil?

Pedro: Infantil: R\$ 149,90. Posso ajudar em mais alguma coisa?

David Capulot:

Nesse caso, o nosso programa encontrou o produto "camisa" e, em seguida, o modelo "infantil", e respondeu apenas com o valor pedido.

Interação 2



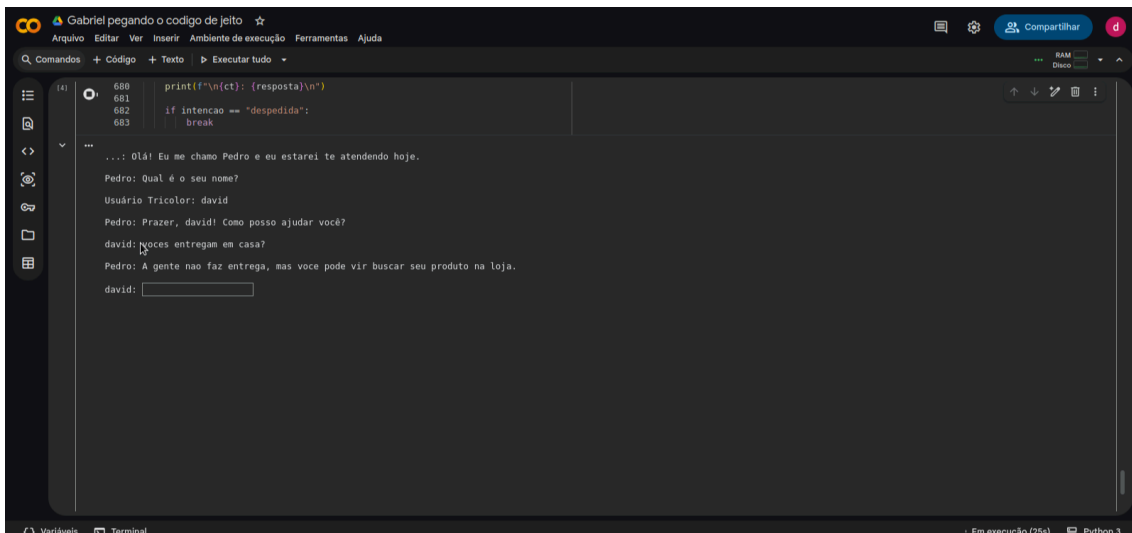
The screenshot shows a Jupyter Notebook with a Python script and its output. The script is as follows:

```
672 entrada = input(f'(mm): ')
673
674
675 intencao = prever_intencao(entrada)
676 resposta, contexto = gerar_resposta_contextual(intencao, entrada, contexto)
677
678 time.sleep(1)
679
680 print(f'\n(ct): {resposta}\n')
681
682 if intencao == "despedida":
683     break
```

The output shows a conversation between Pedro and David. Pedro asks for the user's name, and David responds with 'David capulot'. Pedro then asks how he can help, and David asks for a shirt. Pedro lists various shirt options with prices. David asks for a title, and Pedro provides the price for the title. David then asks for more help, and Pedro responds with a text input field.

A mensagem não cita nenhum produto. Mesmo assim, o Pedro respondeu corretamente, porque guardou "camisa" como o último produto mencionado.

Interação 3



The screenshot shows a Jupyter Notebook with a Python script and its output. The script is as follows:

```
680 print(f'\n(ct): {resposta}\n')
681
682 if intencao == "despedida":
683     break
```

The output shows a conversation between Pedro and David. Pedro asks for the user's name, and David responds with 'david'. Pedro then asks how he can help, and David asks for delivery. Pedro responds that he doesn't do delivery, but David can search for the product in the store. David then provides a text input field.

Aqui o classificador identificou a intenção de entrega e o programa sorteou uma das respostas disponíveis para ela. Em outra execução, a frase poderia ser diferente, mas a ideia seria a mesma.

5. Limitações

O Pedro só sabe o que nós ensinamos a ele. O vocabulário dele vem inteiramente das frases de treino que escrevemos, então qualquer palavra que não esteja nelas simplesmente passa despercebida. Isso vale para gírias, para nomes de produtos que não cadastramos e para erros de digitação: se o cliente escreve "camiza", o programa não reconhece a palavra e age como se ela não existisse. Quando a mensagem inteira é feita de palavras desconhecidas, ele não tem o que analisar e cai na resposta de que não entendeu.

Outra limitação vem do método que escolhemos. Como o *bag of words* só conta as palavras e não olha a ordem em que aparecem, "preço da camisa" e "camisa preço" são a mesma coisa para o Pedro, e ele também não percebe a diferença entre uma pergunta e uma negação. Além disso, o classificador sempre escolhe uma intenção quando reconhece ao menos uma palavra, mesmo que esteja pouco confiante. Em frases ambíguas, isso pode resultar em uma resposta que não tem nada a ver com a dúvida do cliente.

O tratamento do nome do usuário também é frágil. No início da conversa, o Pedro pergunta o nome do cliente e aceita qualquer coisa que seja digitada, inclusive uma frase inteira, sem verificar se aquilo se parece com um nome. Se o cliente responder "quero comprar uma camisa", por exemplo, o programa passará a chamá-lo assim durante toda a conversa. Os nomes com caracteres especiais, como os apelidos de jogadores (um nome de *gamer* com símbolos e letras decorativas), também podem não aparecer corretamente na tela, porque o terminal nem sempre consegue exibir esses caracteres. Só quando o campo fica vazio o programa usa o nome padrão "Torcedor".

O Pedro também tem pouca noção de si mesmo e da própria conversa. Se o cliente perguntar o nome dele, como em "qual seu nome", o programa não responde à pergunta, pois essa frase está no nosso conjunto de exemplos da intenção desconhecida e o bot apenas pede que a pessoa reformule. Da mesma forma, ele não repete nenhuma informação quando o cliente pede, com mensagens do tipo "repete" ou "pode repetir o preço". Esse tipo de pedido não é uma das nossas intenções, e o programa guarda somente o último produto mencionado, e não a última resposta dada. O resultado é que o cliente recebe uma resposta genérica de que o bot não entendeu, quando bastaria repetir o que acabou de dizer.

A parte das regras também tem seus limites. O contexto guarda só um produto por vez, e, se o cliente citar dois na mesma mensagem, o Pedro responde apenas sobre o primeiro que encontrar no catálogo. Os preços que usamos são valores de exemplo, e não os de uma loja real. E o Pedro não faz o que um atendente de verdade faria: ele não consulta estoque, não registra pedidos e não fala de nada que esteja fora do catálogo e das intenções que cadastramos.

6. Considerações finais

O que mais nos deu trabalho foi a parte do contexto. Queríamos que o Pedro entendesse perguntas curtas, como "e a infantil?", mas cada comportamento novo parecia pedir mais um `if` dentro da função de resposta, e o código estava ficando cada vez mais cheio de caminhos. A solução que encontramos foi mostrar tudo de uma só vez quando o cliente cita um produto, com descrição e preços, e guardar apenas o último produto na memória. Assim, o código ficou menor, e a conversa continuou natural. Outra dificuldade foi montar um conjunto de exemplos equilibrado, porque frases parecidas em intenções diferentes, como preço e pagamento, confundiam o modelo.

Se tivéssemos mais tempo, faríamos três mudanças. Primeiro, nós aumentamos bastante o conjunto de treino, com mensagens escritas por pessoas de fora do grupo, para cobrir mais jeitos de perguntar a mesma coisa. Segundo, o nosso grupo iria medir o desempenho do modelo com métricas como acurácia e matriz de confusão, em vez de apenas testar na mão. Terceiro, trocaríamos a longa sequência de `if/elif` da função `gerar_resposta` por um dicionário que liga cada intenção à sua lista de respostas, o que deixaria o código mais curto e mais fácil de ampliar.

```
def gerar_resposta(intencao):
    if intencao == "saudacao":
        resposta = random.choice(respostas_saudacao)
    elif intencao == "ajuda":
        resposta = random.choice(respostas_ajuda)

    elif intencao == "informacao":
        resposta = random.choice(respostas_informacao)

    elif intencao == "produto":
        resposta = random.choice(respostas_produto)

    elif intencao == "preco":
        resposta = random.choice(respostas_preco)

    elif intencao == "pagamento":
        resposta = random.choice(respostas_pagamento)

    elif intencao == "entrega":
        resposta = random.choice(respostas_entrega)

    elif intencao == "despedida":
        resposta = random.choice(respostas_despedida)

    else:
        resposta = random.choice(respostas_desconhecida)

    return resposta
```

Declaração de uso de IA

Durante o desenvolvimento deste trabalho, utilizamos ferramentas de Inteligência Artificial como apoio, a saber, o ChatGPT e o Claude. A ideia do projeto, a escolha das intenções, o catálogo, o fluxo de decisão e a lógica de contexto foram definidos por nós, e o código foi testado e ajustado pelo grupo. Registramos a seguir, de forma transparente, em que partes recebemos ajuda.

Código: Pedimos ao ChatGPT uma versão completa da função `normalizar`, na parte que remove a pontuação com `replace()`, porque o nosso código estava apresentando erro. Também pedimos a ele a criação do conjunto de palavras ignoradas (`palavras_ignorar`), que depois pedimos para escrever sem acentos, para combinar com a normalização do texto.

Listas de respostas. Solicitamos ao ChatGPT a primeira versão das listas de respostas de saudação, ajuda, informação, despedida e desconhecida, com dez frases cada, escritas de maneira natural e sem emojis. Depois, escrevemos e ajustamos as demais listas (produto, preço, pagamento e entrega) e alteramos frases para refletir a regra da nossa loja, em que tudo é presencial.

Conjunto de dados de treino: Pedimos ao ChatGPT ajuda para estruturar o conjunto de dados em duas partes, textos e intenções, e para incluir as intenções de pagamento, produto e entrega. Em seguida, pedimos a ampliação para vinte frases por intenção e um refinamento para separar melhor as intenções de informação e de produto, que tinham frases parecidas. A ferramenta também nos orientou sobre a importância de ter exemplos distintos para cada intenção, por causa do funcionamento do Naive Bayes.

Adaptação do código final: As sugestões recebidas serviram como ponto de partida. A versão final do código difere delas em vários pontos: organizamos os exemplos em um dicionário de frases por intenção, usamos o `CountVectorizer` em vez do TF-IDF sugerido em uma das conversas porque achamos mais fácil o uso, pelo menos o que vimos no slide da aula 4, acrescentamos a intenção de preço e criamos a função de contexto, que não foi sugerida por nenhuma ferramenta.

Aleatório: O nosso grupo usou um compilador online para pegar a formatação que ele deixa o código se copiado de lá, foi esse aqui <https://www.programiz.com/python-programming/online-compiler/> so para caso fica em duvida da formatação usadas algumas vezes nesse documento para o codigo